Projet Personnel : réalisation d'une vidéo

G. Talagas / A. Bayraktar / T. Bracco

Avril 2025

Table des matières

1 Présentation générale du projet **2**

2 Explication de la réalisation d'un logiciel de reconnaissance musicale **2**
2.1 Fonctions d'analyses du signal 2
2.2 Création d'une base de données 5
2.3 Détection d'un morceau inconnu 5
2.4 Performances du modèle 5

3 Explication mathématique de la transformation de Fourier **6**
3.1 Création du logiciel de reconnaissance musicale 7

Table des figures

1 Spectrogramme en amplitude de la chanson "Ramenez la coupe à la maison" de vegedream. 3
2 Représentation schématique d'un hash 3
3 Capture d'écran de la fonction générant les hashes et l'ancrage temporel associé. . 5
4 Performance du système en fonction de la durée de l'extrait 5

1 Présentation générale du projet

Le thème retenu cette année étant **"Période et fréquence"**, nous avons choisi de concentrer notre projet sur la **transformée de Fourier**, un outil mathématique fondamental étroitement lié à la notion de **période** et de **fréquence**. Nous présenterons cette dernière mathématiquement et à travers une des ses diverses applications en développant un logiciel de reconnaissance musical.

2 Explication de la réalisation d'un logiciel de reconnaissance musicale

Nous avons, pour illustrer la transformée de Fourier et ses applications, développé un logiciel de reconnaissance musicale en nous inspirant du fonctionnement de Shazam. Pour cela, nous avons consulté divers articles expliquant son fonctionnement afin de comprendre comment structurer notre code. Nous avons aussi consulté la documentation de divers bibliothèques python et en particulier la documentation de la bibliothèque **Librosa**, qui nous a été très utile. Les résultats du logiciel sont concluants. Ce dernier est capable d'activer le micro de l'ordinateur de récupérer le son pendant deux secondes et de donner le titre de la piste jouée dans la pièce. Le code est structuré de la manière suivante : quatre scripts principaux : traitement audio, génération d'empreintes,

création de base, reconnaissance.

2.1 Fonctions d'analyses du signal

Dans ce fichier nous avons regroupé l'intégralité des fonctions utilisées pour analyser les fichier audio. Nous appelons ensuite ces fonctions dans d'autres fichiers.

On télécharge le fichier audio au format **wav** [1] et on le normalise afin que toutes les amplitudes qu'il contient soient comprises entre 0 et 1. Cela permet de simplifier la comparaison des signaux.

On découpe ensuite le signal en fenêtres/frames et on calcule la STFT (transformée de fourier à court terme) sur chacune des fenêtres. Tout cela est fait par une unique fonction de la bibliothèque **Librosa** qui nous renvoie le spectrogramme complexe du signal. En tant que tel, ce dernier n'est pas exploitable car il contient des informations dont nous n'avons pas besoin comme le déphasage. Nous prenons donc la valeur absolue de ce dernier afin de ne récupérer que l'amplitude. On obtient donc un spectrogramme en amplitude comme celui présenté en figure 1.

1. Les fichiers .wav ne contiennent pas des fréquences mais des amplitudes qui servent à dire au haut parleur comment déplacer sa membrane. C'est l'amplitude du mouvement de la membrane.

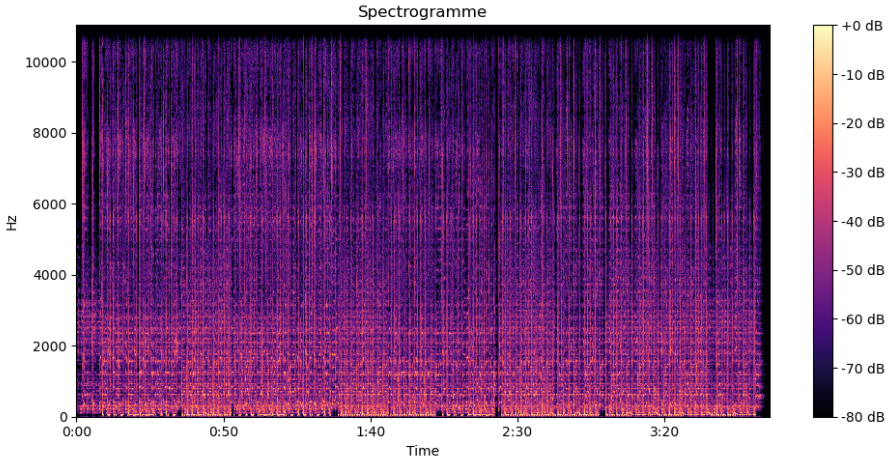


FIGURE 1 – Spectrogramme en amplitude de la chanson "Ramenez la coupe à la maison" de vegedream.

L'analyse du signal peut alors commencer. La méthode que nous appliquons est la suivante :

Nous récupérons les n fréquences [2] ayant la plus forte intensité pour chaque fenêtre, ce sont les pics. Ensuite, notre première approche consistait à créer une empreinte pour chaque son, c'est à dire un tableau contenant tous les pics et le temps auquel ils avaient lieu. Cette méthode s'est avérée efficace mais peu précise car ne marchant pas pour des extraits commençant après le début du morceau. Nous avons alors opté pour une méthode plus robuste qui est aussi celle mise en place par Shazam : nous avons crée des "hashs". Un hash est (en général [3]) un trouple :

$$(f_1, f_2, \Delta t)$$

Avec :

— f_1 la fréquence d'un premier pic.
— f_2 la fréquence d'un second pic présent dans une fenêtre suivante.
— Δt le temps s'étant écoulé entre ces deux pics.

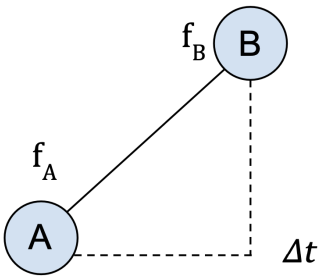


FIGURE 2 – Représentation schématique d'un hash

La création de hashs est très utile car elle permet de s'affranchir de la dépendance temporelle : elle permet d'encoder des motifs indépendamment du moment ou ils apparaissent dans l'extrait.

2. Dans notre cas ce sera 5.
3. Dans notre cas nous ne constituons que des hashs de trois valeurs. Inclure plus de valeurs dans chaque hash permettrait d'encoder des motifs plus complexes et d'avoir un meilleur taux de réussite lors des reconnaissances de sons. Cependant cela ne nous est pas utile car le modèle est déja redoutablement efficace.

Naturellement, plus nous incluons de fréquences dans notre hash plus ce dernier contiendra des motifs complexes qui permettront de reconnaître avec plus de précision les pistes jouées. Néanmoins la création de hash demande une puissance de calcul considérable (car le nombre de hash à calculer est considérable). En effet, en posant N le nombre total de fenêtres, n le nombre de pics dans chaque fenêtre, et $t_\ell,\ t_i$ les temps auxquels commencent respectivement les fenêtres ℓ et i, on peut décrire l'ensemble des hashs générés pour chaque piste par :

$$\bigcup_{i=1}^{N} \bigcup_{\ell=i+1}^{\min(i+5,\ N)} \bigcup_{k=1}^{n} \bigcup_{m=1}^{n} \left\{ \left(f_k^i,\ f_m^\ell,\ \Delta t = t_\ell - t_i \right) \right\}$$

Cette formule symbolise l'idée que l'on formera (pour chaque fenêtre) tous les couples de pics possibles entre la fenêtre actuelle et les 5 fenêtre suivants (sans oublier le temps entre les pics du couple).Le nombre de hashs formé est donc colossal : pour un morceau de 2min30 nous créons **700 000 hashs**.

Cependant, bien que déjà efficace, le hash tout seul à quelques faiblesses. En effet sans informations additionnelles le programme pourrait confondre des morceaux qui présentent les mêmes hashs dans un ordre totalement différent. Il est donc important de coupler chaque hash à un **ancrage temporel**. L'ancrage temporel renseigne simplement sur le temps écoulé entre le début de l'enregistrement et l'apparition de la fréquence f_1 première fréquence de notre hash. Maintenant que nous avons l'ancrage temporel nous pouvons constituer l'empreinte de chaque piste. Cette dernière contient les couples (hash,ancrage temporel) ce qui donne en écrivant A l'ancrage temporel :

$$\text{empreinte} = [((f_1, f_2, \Delta t), \mathbf{A}), ((f_1, f_2, \Delta t), \mathbf{A'}), ..., ((f_n, f_m, \Delta t), \mathbf{A''})]$$

Vient alors l'étape de comparaison du signal test aux signaux connus. Pour cela nous récupérons l'empreinte du signal test. La première étape de la comparaison consiste à vérifier si les hashs présents dans l'empreinte du son testé le sont aussi dans les empreintes des fichiers connus. Si c'est le cas on passe à la comparaison des ancrages temporels, on calcule la différence entre l'encrage temporel du hash de la base et du hash de l'extrait testé :

$$\Delta_{\text{ancrage}} = \text{ancrage-base} - \text{ancrage-extrait}$$

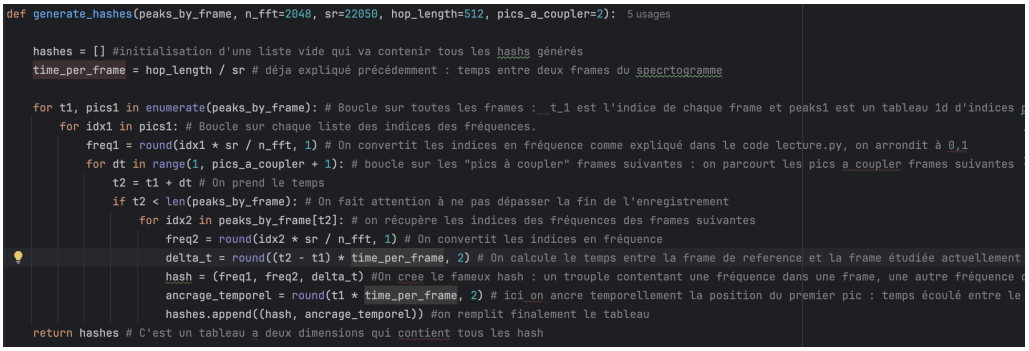
On crée alors un tableau à une dimension pour chaque morceau de la base contenant tous les décalages temporels entre les hashs du morceau de la base et du morceau testé. On trouve ensuite le décalage qui est le plus présent dans le tableau et on ajoute le nombre d'occurrence de ce décalage comme score au morceau. On répète ensuite ce processus pour chaque morceau. Le morceau ayant le plus haut score est celui qui est le plus à même de correspondre au morceau testé. En effet, les morceaux ayant un faible score sont :

— Soit les morceaux qui ont peu de hashs en commun avec le morceau test, qui par conséquent un petit tableau de décalage temporel et donc peu d'occurrences du même décalage $\implies$ faible score [4].

— Soit ceux qui ont de nombreux hashs en commun mais peu de décalages communs et qui du coup ont aussi un faible score.

On comprend donc qu'un score élevé signifie à la fois un décalage temporel cohérent et la présence des mêmes motifs sonores. On affiche donc le titre ayant le meilleur score.

4. Le score reflète donc bien à la fois le décalage temporel et la présence des mêmes motifs sonores.



```
def generate_hashes(peaks_by_frame, n_fft=2048, sr=22050, hop_length=512, pics_a_coupler=2):  5 usages

    hashes = [] #initialisation d'une liste vide qui va contenir tous les hashs générés
    time_per_frame = hop_length / sr # déja expliqué précédemment : temps entre deux frames du spectrogramme

    for t1, pics1 in enumerate(peaks_by_frame): # Boucle sur toutes les frames : _t_1 est l'indice de chaque frame et peaks1 est un tableau 1d d'indices p
        for idx1 in pics1: # Boucle sur chaque liste des indices des fréquences.
            freq1 = round(idx1 * sr / n_fft, 1) # On convertit les indices en fréquence comme expliqué dans le code lecture.py, on arrondit à 0,1
            for dt in range(1, pics_a_coupler + 1): # boucle sur les "pics à coupler" frames suivantes : on parcourt les pics a coupler frames suivantes
                t2 = t1 + dt # On prend le temps
                if t2 < len(peaks_by_frame): # On fait attention à ne pas dépasser la fin de l'enregistrement
                    for idx2 in peaks_by_frame[t2]: # on récupère les indices des fréquences des frames suivantes
                        freq2 = round(idx2 * sr / n_fft, 1) # On convertit les indices en fréquence
                        delta_t = round((t2 - t1) * time_per_frame, 2) # On calcule le temps entre la frame de reference et la frame étudiée actuellement
                        hash = (freq1, freq2, delta_t) #On cree le fameux hash : un trouple contentant une fréquence dans une frame, une autre fréquence d
                        ancrage_temporel = round(t1 * time_per_frame, 2) # ici on ancre temporellement la position du premier pic : temps écoulé entre le
                        hashes.append((hash, ancrage_temporel)) #on remplit finalement le tableau
    return hashes # C'est un tableau a deux dimensions qui contient tous les hash
```

FIGURE 3 – Capture d'écran de la fonction générant les hashes et l'ancrage temporel associé.

2.2 Création d'une base de données

Afin de pouvoir retrouver les sons écoutés il nous est nécessaire de constituer une base de données. Cette étape est relativement simple car cela consiste plus ou moins à créer un tableau 2D contenant dans une colonne le titre du morceau et dans l'autre son empreinte calculée à l'aide des fonctions écrites précédemment.

2.3 Détection d'un morceau inconnu

L'acquisition du son de l'ordinateur se fait grâce aux fonctions intégrées des bibliothèques **sounddevice** et **soundfile**. On récupère ensuite l'empreinte du son fraîchement enregistré et on la compare avec la base de données.

2.4 Performances du modèle

Afin de quantifier la fiabilité du modèle de reconnaissance musical crée nous avons procédé de la manière suivante : Nous avons constitué une base de données de 6 sons. Cela est relativement peu mais avoir une base de donnée plus grande ralentit considérablement le processus de reconnaissance. Ensuite nous avons prélevé 10 extraits de chaque morceau de manière aléatoire pour des durées de [0.1,0.2,...,1] secondes. Nous avons ensuite demandé au logiciel de comparer chacun des extraits à la base de donnée et de prédire le morceau joué. Nous avons regroupé les résultats obtenus sur le graphique suivant :

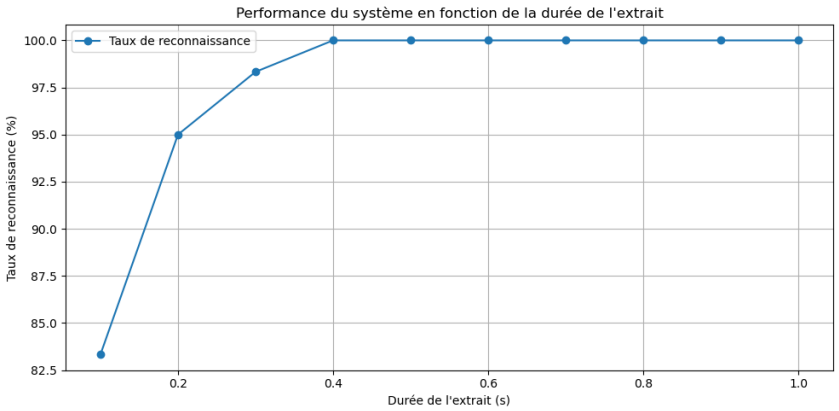


FIGURE 4 – Performance du système en fonction de la durée de l'extrait

Nous pouvons observer sur la figure 4 que même pour des extraits très courts le programme arrive à trouver avec précision les musiques jouées. En effet pour des extraits de 0.1 sec le modèle affiche

plus de 80% de bonnes réponses sur les 60 extraits qu'il a eu à identifier. A partir de 0,4 seconde d'extrait le modèle le modèle est capable d'identifier avec certitude le morceau joué : il a su identifier correctement les 60 extraits qui lui ont été fournis. Bien entendu cette méthode d'évaluation des performances n'est pas parfaite. D'une part nous fournissons des extraits "parfaits" au programme, ces derniers ne sont pas pollués par quelconque bruit extérieurs car ils sont tout simplement extraits de la base de donnée. Le modèle est néanmoins capable d'identifier correctement les sons pollués mais nous ne sommes pas en mesure de quantifier la précision de ce dernier dans de telles conditions. L'impressionnante précision des résultats obtenus démontre l'efficacité redoutable de la pierre angulaire de notre programme : la transformation de Fourier dans l'analyse des signaux sonores et la décomposition de signaux quelconques en somme de sinusoïdes.

3 Explication mathématique de la transformation de Fourier

— Partir de la décomposition d'une fonction périodique $f(t)$ en série de Fourier :

$$f(t) = \sum_{n=-\infty}^{+\infty} c_n \, e^{2\pi i \, \frac{n}{T} t}, \quad c_n = \frac{1}{T} \int_{-T/2}^{T/2} f(s) e^{-2\pi i \, \frac{n}{T} s} \, ds.$$

— Expliquer que T est la période du signal et fixe l'intervalle d'intégration ainsi que l'espacement des fréquences ($\xi_n = \frac{n}{T}$, avec $\Delta\xi = \frac{1}{T}$).

— Interpréter les coefficients c_n comme le "poids" (amplitude et phase) de chaque composante sinusoïdale.

— Montrer que lorsque $T \to \infty$, la somme devient une intégrale continue, aboutissant à la transformée de Fourier :

$$F(\xi) = \int_{-\infty}^{+\infty} f(t) e^{-2\pi i \, t \, \xi} \, dt.$$

— Présenter la formule d'inversion continue :

$$f(t) = \int_{-\infty}^{+\infty} F(\xi) e^{2\pi i \, t \, \xi} \, d\xi.$$

— Demonstration principale : Processus de passage de la version continue à la version discrète. Pour l'application numérique, tronquer $f(t)$ sur un intervalle fini et l'échantillonner en N points ($\Delta t = T/N$).Remplacer l'intégrale par une somme de Riemann et discrétiser les fréquences ($\xi_k = \frac{k}{T}$), conduisant à la définition de la DFT :

$$X[k] = \sum_{n=0}^{N-1} x[n] \, e^{-2\pi i \, \frac{nk}{N}}.$$

— Expliquer que la DFT fournit une approximation du spectre continu,

— Explication brève de la FFT (fast fourier transform) .

Références bibliographiques

3.1 Création du logiciel de reconnaissance musicale

1. Wang, A. (2003). An Industrial Strength Audio Search Algorithm. [En ligne]. Disponible : https://www.ee.columbia.edu/~dpwe/papers/Wang03-shazam.pdf
2. Wikipedia contributors. Acoustic fingerprint. https://en.wikipedia.org/wiki/Acoustic_fingerprint
3. Wikipedia contributors. Short-time Fourier transform. https://en.wikipedia.org/wiki/Short-time_Fourier_transform
4. Librosa. STFT – Short-Time Fourier Transform. Documentation officielle. https://librosa.org/doc/main/generated/librosa.stft.html
5. Macleod.C. Abracadabra How does Shazam work, 2022. https://www.cameronmacleod.com/blog/how-does-shazam-work
6. Python SoundDevice Documentation. https://python-sounddevice.readthedocs.io